

Mini-Projet Réseaux TP

Monitoring Réseau Client–Serveur avec Sockets TCP & UDP

Étudiant(s) : Hamdi Mohamed Aziz
Maddouri Ferdaous

Filière : RT2

Enseignant : Dr. Kamel Karoui

Table des matières

1	Introduction	2
2	Architecture du Système	2
2.1	Vue d'ensemble	2
2.2	Côté Serveur (Collecteur)	2
2.3	Côté Client (Agent)	3
3	Description du Protocole	3
3.1	Messages Client → Serveur	3
3.2	Réponses Serveur → Client	3
3.3	Contraintes du Protocole	3
4	Choix Techniques	4
4.1	Langage et bibliothèques	4
4.2	Gestion de la concurrence	4
4.3	Détection des agents inactifs	4
5	Code Source	5
5.1	Serveur TCP — <code>serveurTCP.py</code>	5
5.2	Serveur UDP — <code>serveurUDP.py</code>	5
5.3	Client TCP — <code>clientTCP.py</code>	5
5.4	Client UDP — <code>clientUDP.py</code>	5
5.5	Client Personnalisé — <code>customClient.py</code>	5
5.6	Simulation de Flood — <code>flood.py</code>	5
6	Imprimés Écrans du Fonctionnement	5
6.1	Connexion d'un seul client	6
6.2	Connexion de plusieurs clients simultanément	6
6.3	Envoi d'un message mal formé	7
6.4	Arrêt propre d'un client	7
6.5	Arrêt brutal d'un client	8
6.6	Arrêt du serveur	8
6.7	Tentative de connexion à un serveur inactif	8
6.8	Calcul des moyennes	9
7	Difficultés Rencontrées	9
8	Extensions Réalisées	11
8.1	Ajout d'un mode UDP	11
8.2	Détection d'agents inactifs	13
8.3	Export des statistiques en CSV	13
8.4	Ajout d'un identifiant unique UUID	14
8.5	Simulation d'attaque — Envoi massif de REPORT	14
8.5.1	Analyse du comportement du serveur face au flood	16
8.6	Sécurité — Ownership check	16
9	Conclusion	17

1. Introduction

Ce mini-projet consiste à implémenter une application de monitoring réseau distribuée basée sur une architecture client–serveur. Chaque client (agent) collecte périodiquement des métriques locales (CPU et mémoire) et les envoie à un serveur central. Le serveur agrège ces données, maintient l'état des agents actifs et affiche des statistiques globales via une interface graphique Tkinter.

Le projet est réalisé en Python en utilisant les bibliothèques standards ainsi que `psutil` pour la collecte des métriques système. Deux protocoles de transport sont supportés : **TCP** (port 5000) et **UDP** (port 5001). Des extensions ont également été développées : une simulation d'attaque par flood, des mécanismes de protection associés, un export CSV automatique et une suite de tests automatisée.

2. Architecture du Système

2.1. Vue d'ensemble

L'application repose sur une architecture client–serveur classique. Le serveur joue le rôle de collecteur central, tandis que chaque client constitue un agent de monitoring autonome. Deux modes de communication sont supportés :

- **TCP** sur le port 5000 — fiable, orienté connexion, chaque client est géré dans un thread dédié.
- **UDP** sur le port 5001 — sans connexion, plus léger, avec mécanisme de retry côté client.

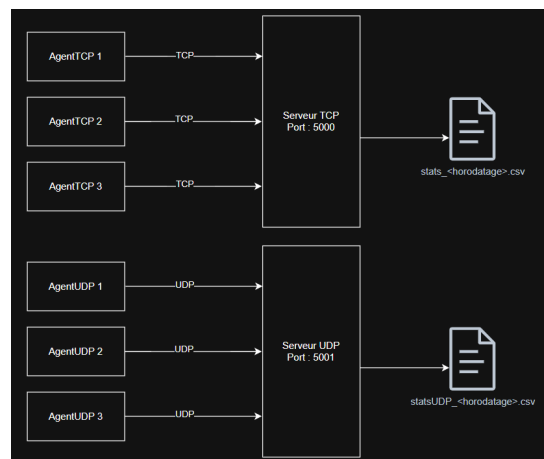


FIGURE 1 – Schéma de l'architecture client–serveur

2.2. Côté Serveur (Collecteur)

Le serveur (TCP et UDP) est conçu pour :

- Accepter plusieurs connexions/datagrammes simultanés grâce au multithreading ;
- Gérer chaque client TCP dans un thread séparé ; le serveur UDP utilise un thread de réception unique avec une table `addr_to_agent` ;
- Maintenir la liste des agents actifs dans un dictionnaire partagé protégé par `threading.Lock` ;

- Calculer et afficher périodiquement les statistiques globales dans une fenêtre Tkinter (mise à jour via **after(0, ...)** pour la thread-safety) ;
- Exporter automatiquement un snapshot toutes les T secondes dans un fichier CSV horodaté ;
- Ne jamais s'arrêter en cas d'erreur d'un client individuel.

2.3. Côté Client (Agent)

Chaque agent est conçu pour :

- Se connecter au serveur et s'enregistrer via un message **HELLO** ;
- Collecter le pourcentage CPU et la mémoire RAM utilisée via **psutil** ;
- Envoyer un rapport toutes les T secondes via un message **REPORT** ;
- Se déconnecter proprement via un message **BYE** (envoyé automatiquement à **Ctrl+C**) ;
- En UDP : réessayer le **HELLO** jusqu'à **MAX_RETRIES = 5** fois si aucune réponse n'est reçue.

3. Description du Protocole

3.1. Messages Client → Serveur

Message	Format
Enregistrement	HELLO <agent_id> <hostname>
Rapport	REPORT <agent_id> <timestamp> <cpu_pct> <ram_mb>
Déconnexion	BYE <agent_id>

TABLE 1 – Messages envoyés par le client

3.2. Réponses Serveur → Client

Réponse	Signification
OK	Message reçu et traité avec succès
ERROR AlreadyRegistered	L'adresse/connexion a déjà envoyé un HELLO
ERROR AgentIDTaken	L'identifiant est déjà utilisé par un autre agent
ERROR Max agents reached	Le plafond MAX_AGENTS est atteint
ERROR Unauthorized	Tentative d'opération sur un agent non possédé
ERROR Rate limit	REPORT envoyé trop rapidement
ERROR InvalidData	Valeurs CPU/RAM invalides ou hors limites
ERROR InvalidMessageFormat	Message inconnu ou mal formé

TABLE 2 – Réponses envoyées par le serveur

3.3. Contraintes du Protocole

- **agent_id** : sans espaces, unique par connexion/adresse.

- **cpu_pct** : réel entre 0 et 100.
- **ram_mb** : réel supérieur ou égal à 0.
- Un agent est considéré actif si un **REPORT** est reçu dans une fenêtre de $3 \times T$ secondes (**MAX_AGE** = 15 avec **T** = 5).
- Un délai minimum de **RATE_LIMIT_S** = 2 secondes est imposé entre deux **REPORT** d'un même agent.
- Le nombre d'agents simultanés est plafonné à **MAX_AGENTS** = 30.
- En TCP, seule la connexion propriétaire peut envoyer des **REPORT** ou **BYE** pour son agent (*ownership check*).
- En UDP, seule l'adresse source ayant envoyé le **HELLO** peut opérer sur l'agent (table **addr_to_agent**).

4. Choix Techniques

4.1. Langage et bibliothèques

Le projet a été implémenté en Python 3 en utilisant :

- **socket** : communication TCP/UDP ;
- **threading** : gestion des connexions simultanées ;
- **psutil** : collecte des métriques CPU et RAM ;
- **uuid** : génération d'identifiants uniques ;
- **csv**, **time**, **datetime** : export et horodatage ;
- **tkinter** : interface graphique du tableau de bord.

4.2. Gestion de la concurrence

Un verrou (**threading.Lock**) protège le dictionnaire partagé **agents** contre les accès concurrents. En TCP, chaque connexion est traitée dans un thread indépendant. En UDP, un thread unique reçoit tous les datagrammes ; la table **addr_to_agent** joue le rôle d'équivalent de **current_agent_id** pour identifier le propriétaire de chaque message. Les modifications du widget Tkinter depuis les threads secondaires se font via **text_widget.after(0, callback)** afin de garantir l'exécution dans le thread principal de l'interface graphique.

4.3. Détection des agents inactifs

Un agent est retiré des statistiques si aucun **REPORT** n'est reçu dans une fenêtre de $3 \times T$ secondes. Le thread **handle_stats**, qui se réveille toutes les T secondes, vérifie le champ **last_seen** de chaque agent et génère une alerte explicite avant de le supprimer. Seuls les agents ayant reçu au moins un **REPORT** valide (**verified** = **True**) sont inclus dans les moyennes affichées.

5. Code Source

5.1. Serveur TCP — `serveurTCP.py`

Le serveur TCP intègre : gestion multi-clients par threading (`handle_client`), validation complète du protocole avec ownership check, rate-limiting, détection des agents inactifs, calcul des moyennes, export CSV horodaté et tableau de bord Tkinter.

5.2. Serveur UDP — `serveurUDP.py`

Le serveur UDP reprend la même logique avec les adaptations inhérentes à l'absence de connexion : table `addr_to_agent` pour l'ownership, réception dans un thread unique (`listening_loop`), handlers dédiés `handle_hello`, `handle_report`, `handle_bye`.

5.3. Client TCP — `clientTCP.py`

Le client TCP génère un **AGENT_ID** de la forme **Agent_<8 car. UUID>**, envoie **HELLO** puis boucle sur **REPORT** toutes les T secondes. **Ctrl+C** déclenche l'envoi propre du **BYE**.

5.4. Client UDP — `clientUDP.py`

Identique au client TCP dans son comportement, avec en plus un mécanisme de retry sur le **HELLO** initial (**MAX_RETRIES** = 5, délai de 2 secondes), un timeout socket (**TIMEOUT** = 3) et la gestion de **ConnectionResetError** (WinError 10054).

5.5. Client Personnalisé — `customClient.py`

Permet d'envoyer un message arbitraire saisi interactivement en boucle. Après **MAX_TRIES** = 5 rejets consécutifs, le client abandonne automatiquement.

5.6. Simulation de Flood — `flood.py`

Lance 50 threads simultanés, chacun envoyant 1000 messages **REPORT** en rafale pour tester le rate-limiting et la limite **MAX_AGENTS** = 30.

6. Imprimés Écrans du Fonctionnement

Tous les scénarios ci-dessous ont été exécutés via `run_tests.bat`

6.1. Connexion d'un seul client

```

[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-10-2026_0155.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 54462)
[HELLO] Agent 'Agent_1ff2e370' (TCP-Client-ONE) registered.
[REPORT] 'Agent_1ff2e370' → CPU: 10.20% RAM: 15695.59 MB
[REPORT] 'Agent_1ff2e370' → CPU: 16.80% RAM: 15939.37 MB
[REPORT] 'Agent_1ff2e370' → CPU: 8.60% RAM: 15869.31 MB
[REPORT] 'Agent_1ff2e370' → CPU: 10.00% RAM: 15678.49 MB

Agent Monitor Dashboard "SocketMaster"

Live — current snapshot
Last update : 01:55:51
Active agents : 1
Avg CPU : 10.00%
Avg RAM : 15678.49 MB

Agent_1ff2e370 (TCP-Client-ONE)
CPU : 10.00% | RAM : 15678.49 MB

History — all past snapshots
[01:55:36] Agents: 1 | Avg CPU: 10.20% | Avg RAM: 15695.5
Agent_1ff2e370 (TCP-Client-ONE) CPU: 10.20% RAM: 15695.5
[01:55:41] Agents: 1 | Avg CPU: 16.80% | Avg RAM: 15939.3
Agent_1ff2e370 (TCP-Client-ONE) CPU: 16.80% RAM: 15939.3
[01:55:46] Agents: 1 | Avg CPU: 8.60% | Avg RAM: 15869.3
Agent_1ff2e370 (TCP-Client-ONE) CPU: 8.60% RAM: 15869.31
[01:55:51] Agents: 1 | Avg CPU: 10.00% | Avg RAM: 15678.4
Agent_1ff2e370 (TCP-Client-ONE) CPU: 10.00% RAM: 15678.4

```

FIGURE 2 – Un seul agent connecté, envoi de rapports périodiques (test 1)

6.2. Connexion de plusieurs clients simultanément

```

[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-09-2026_1924.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 55843)
[HELLO] Agent 'Agent_b7d8b12c' (TCP-Client-ONE) registered.
[REPORT] 'Agent_b7d8b12c' → CPU: 13.90% RAM: 14915.98 MB
[+] Connection from ('127.0.0.1', 55844)
[HELLO] Agent 'Agent_dc16ac36' (TCP-Client-TWO) registered.
[REPORT] 'Agent_dc16ac36' → CPU: 12.70% RAM: 15007.11 MB
[+] Connection from ('127.0.0.1', 55845)
[HELLO] Agent 'Agent_aab45a73' (TCP-Client-THREE) registered.
[REPORT] 'Agent_aab45a73' → CPU: 15.70% RAM: 15026.36 MB
[REPORT] 'Agent_b7d8b12c' → CPU: 8.60% RAM: 15039.47 MB
[REPORT] 'Agent_dc16ac36' → CPU: 18.40% RAM: 15047.02 MB
[REPORT] 'Agent_aab45a73' → CPU: 15.40% RAM: 15038.02 MB
[REPORT] 'Agent_b7d8b12c' → CPU: 6.40% RAM: 15038.32 MB
[REPORT] 'Agent_dc16ac36' → CPU: 4.80% RAM: 15022.98 MB
[REPORT] 'Agent_aab45a73' → CPU: 6.80% RAM: 15024.32 MB
[REPORT] 'Agent_b7d8b12c' → CPU: 6.50% RAM: 15047.95 MB

Agent Monitor Dashboard "SocketMaster"

Live — current snapshot
Last update : 01:55:51
Active agents : 3
Avg CPU : 10.00%
Avg RAM : 15678.49 MB

Agent_1ff2e370 (TCP-Client-ONE)
CPU : 10.00% | RAM : 15678.49 MB

History — all past snapshots
[01:55:36] Agents: 1 | Avg CPU: 10.20% | Avg RAM: 15695.5
Agent_1ff2e370 (TCP-Client-ONE) CPU: 10.20% RAM: 15695.5
[01:55:41] Agents: 1 | Avg CPU: 16.80% | Avg RAM: 15939.3
Agent_1ff2e370 (TCP-Client-ONE) CPU: 16.80% RAM: 15939.3
[01:55:46] Agents: 1 | Avg CPU: 8.60% | Avg RAM: 15869.3
Agent_1ff2e370 (TCP-Client-ONE) CPU: 8.60% RAM: 15869.31
[01:55:51] Agents: 1 | Avg CPU: 10.00% | Avg RAM: 15678.4
Agent_1ff2e370 (TCP-Client-ONE) CPU: 10.00% RAM: 15678.4

```

FIGURE 3 – Plusieurs agents connectés simultanément, statistiques globales affichées (test 2)

6.3. Envoi d'un message mal formé

```

TCP-Server - python serveurT x + -
[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-09-2026_1928.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 49154)
[HELLO] Agent 'Agent_942f75bc' (TCP-Client-ONE) registered.
[REPORT] 'Agent_942f75bc' → CPU: 20.80% RAM: 15146.38 MB
[REPORT] 'Agent_942f75bc' → CPU: 18.60% RAM: 15009.03 MB
[REPORT] 'Agent_942f75bc' → CPU: 4.28% RAM: 15050.56 MB
[REPORT] 'Agent_942f75bc' → CPU: 6.70% RAM: 15009.45 MB
[+] Connection from ('127.0.0.1', 49156)
[HELLO] Agent 'Agent_2d5cfe4f' (AgentID) registered.
[ERROR] Invalid REPORT data from ('127.0.0.1', 49156): CPU/RAM out of range
[ERROR] Invalid REPORT data from ('127.0.0.1', 49156): CPU/RAM out of range
[REPORT] 'Agent_942f75bc' → CPU: 3.70% RAM: 15025.55 MB
[ERROR] Invalid REPORT data from ('127.0.0.1', 49156): CPU/RAM out of range
[REPORT] 'Agent_942f75bc' → CPU: 6.30% RAM: 14979.94 MB
[ERROR] Invalid REPORT data from ('127.0.0.1', 49156): CPU/RAM out of range
[REPORT] 'Agent_942f75bc' → CPU: 3.40% RAM: 14968.06 MB
[ERROR] Invalid REPORT data from ('127.0.0.1', 49156): CPU/RAM out of range
[INFO] Agent 'Agent_2d5cfe4f' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 49156)

TCP-Client 1 - python clientT x + -
[TCP CLIENT] Starting with ID: Agent_942f75bc | Hostname = TCP-Client-ONE
[TCP CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_942f75bc (CP-Client-ONE)'
[OK] 'REPORT Agent_942f75bc 1775759285 20.80000 15146.38'
[OK] 'REPORT Agent_942f75bc 1775759290 18.60000 15009.03'
[OK] 'REPORT Agent_942f75bc 1775759295 4.28000 15050.56'
[OK] 'REPORT Agent_942f75bc 1775759300 6.70000 15009.45'
[OK] 'REPORT Agent_942f75bc 1775759305 3.70000 15025.55'
[OK] 'REPORT Agent_942f75bc 1775759310 6.30000 14979.94'
[OK] 'REPORT Agent_942f75bc 1775759315 3.40000 14968.06'

Custom Client x + -
Enter your HOSTNAME: agent1
Enter your custom message: REPORT agent1 0 999 1024
[CLIENT] Starting with ID: Agent_2d5cfe4f | Hostname = agent1
[CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_2d5cfe4f agent1.'
[WARNING] 'REPORT agent1 0 999 1024' → server replied: 'ERROR InvalidData'
[CLIENT] Rejected (1/5) - retrying in 5s...
[WARNING] 'REPORT agent1 0 999 1024' → server replied: 'ERROR InvalidData'
[CLIENT] Rejected (2/5) - retrying in 5s...
[WARNING] 'REPORT agent1 0 999 1024' → server replied: 'ERROR InvalidData'
[CLIENT] Rejected (3/5) - retrying in 5s...
[WARNING] 'REPORT agent1 0 999 1024' → server replied: 'ERROR InvalidData'
[CLIENT] Rejected (4/5) - retrying in 5s...
[WARNING] 'REPORT agent1 0 999 1024' → server replied: 'ERROR InvalidData'
[CLIENT] Rejected (5/5) - retrying in 5s...
[ERROR] Server rejected message 5 times in a row. Aborting.
[CLIENT] Exiting.
[CLIENT] Disconnected.
C:\Users\Ferdous\Downloads\files>

```

FIGURE 4 – Le serveur retourne ERROR pour un message invalide (test 3)

6.4. Arrêt propre d'un client

```

TCP-Server - python serveurT x + -
[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-10-2026_0157.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 56022)
[HELLO] Agent 'Agent_3d515572' (TCP-Client-ONE) registered.
[REPORT] 'Agent_3d515572' → CPU: 15.10% RAM: 15993.43 MB
[REPORT] 'Agent_3d515572' → CPU: 10.58% RAM: 15960.18 MB
[REPORT] 'Agent_3d515572' → CPU: 11.20% RAM: 15888.28 MB
[BYE] Agent 'Agent_3d515572' disconnected properly.
[-] Connection closed: ('127.0.0.1', 56022)
No agents are active as of now...
No agents are active as of now...

TCP Client 1 x + -
[TCP CLIENT] Starting with ID: Agent_3d515572 | Hostname = TCP-Client-ONE
[TCP CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_3d515572 TCP-Client-ONE'
[OK] 'REPORT Agent_3d515572 1775782668 15.10000 15993.43'
[OK] 'REPORT Agent_3d515572 1775782673 14.50000 15960.18'
[OK] 'REPORT Agent_3d515572 1775782678 11.20000 15888.28'
[TCP CLIENT] Interrupted by user. Disconnecting properly...
[OK] 'BYE Agent_3d515572'
[TCP CLIENT] Disconnected.
C:\Users\Ferdous\Downloads\files>

Agent Monitor Dashboard "SocketMaster"
Live - current snapshot
Last update : 01:58:06
Active agents : 0
Avg CPU : -
Avg RAM : -

History - all past snapshots
[01:57:51] Agents: 1 | Avg CPU: 15.10% | Avg RAM: 15993.4
Agent_3d515572 (TCP-Client-ONE) CPU: 15.10% RAM: 15993.4
[01:57:56] Agents: 1 | Avg CPU: 14.50% | Avg RAM: 15960.1
Agent_3d515572 (TCP-Client-ONE) CPU: 14.50% RAM: 15960.1
[01:58:01] No agents are active as of now...
[01:58:06] No agents are active as of now...

```

FIGURE 5 – Arrêt propre d'un client via Ctrl+C : envoi du message BYE et déconnexion immédiate

6.5. Arrêt brutal d'un client

```

[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export -> stats_Apr-09-2026_1917.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 52683)
[HELLO] Agent 'Agent_1212bb00' (TCP-Client-ONE) registered.
[REPORT] 'Agent_1212bb00' -> CPU: 15.00% RAM: 16114.45 MB
[+] Connection from ('127.0.0.1', 52683)
[HELLO] Agent 'Agent_64a263e5' (TCP-Client-TWO) registered.
[REPORT] 'Agent_64a263e5' -> CPU: 11.80% RAM: 15979.59 MB
[REPORT] 'Agent_1212bb00' -> CPU: 17.50% RAM: 16004.40 MB
[REPORT] 'Agent_64a263e5' -> CPU: 20.70% RAM: 15994.82 MB
[REPORT] 'Agent_1212bb00' -> CPU: 13.30% RAM: 16043.01 MB
[REPORT] 'Agent_64a263e5' -> CPU: 10.20% RAM: 15993.26 MB
[!] Error with client ('127.0.0.1', 52683): [WinError 10054] An existing connection was forcibly closed
  by the remote host.
[INFO] Agent 'Agent_64a263e5' connection lost - alert in 15s if no reconnection.
[-] Connection closed: ('127.0.0.1', 52683)
[REPORT] 'Agent_1212bb00' -> CPU: 7.80% RAM: 15897.31 MB
[REPORT] 'Agent_1212bb00' -> CPU: 7.70% RAM: 15921.53 MB
[REPORT] 'Agent_1212bb00' -> CPU: 9.90% RAM: 15877.50 MB
[REPORT] 'Agent_1212bb00' -> CPU: 9.20% RAM: 15900.07 MB

[ALERT] Agent 'Agent_64a263e5' DISCONNECTED unexpectedly (silent for 18s)
[INFO] Agent 'Agent_64a263e5' removed from stats.
[REPORT] 'Agent_1212bb00' -> CPU: 5.40% RAM: 15883.15 MB
[REPORT] 'Agent_1212bb00' -> CPU: 7.50% RAM: 15883.78 MB
[REPORT] 'Agent_1212bb00' -> CPU: 11.00% RAM: 15805.90 MB
[REPORT] 'Agent_1212bb00' -> CPU: 10.40% RAM: 15823.37 MB

[TCP CLIENT] Starting with ID: Agent_1212bb00 | Hostname = TCP-Client-ONE
[TCP CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_1212bb00 (TCP-Client-ONE)'
[OK] 'REPORT Agent_1212bb00 1775782794 16.90000 16114.45'
[OK] 'REPORT Agent_1212bb00 1775782799 17.50000 16004.40'
[OK] 'REPORT Agent_1212bb00 1775782804 13.30000 16043.01'
[OK] 'REPORT Agent_1212bb00 1775782809 7.80000 15897.31'
[OK] 'REPORT Agent_1212bb00 1775782814 7.70000 15921.53'
[OK] 'REPORT Agent_1212bb00 1775782819 9.90000 15877.50'
[OK] 'REPORT Agent_1212bb00 1775782824 9.20000 15900.07'
[OK] 'REPORT Agent_1212bb00 1775782829 5.00000 15883.15'
[OK] 'REPORT Agent_1212bb00 1775782834 7.50000 15883.78'
[OK] 'REPORT Agent_1212bb00 1775782839 11.00000 15805.90'
[OK] 'REPORT Agent_1212bb00 1775782844 10.40000 15823.37'

Agent Monitor Dashboard "SocketMaster"

Live - current snapshot
Last update : 02:00:47
Active agents : 1
Avg CPU : 10.40%
Avg RAM : 15823.37 MB

Agent_1212bb00 (TCP-Client-ONE)
CPU : 10.40% | RAM : 15823.37 MB

History - all past snapshots
[02:00:42] Agents: 1 | Avg CPU: 11.80% | Avg RAM: 15805.90 MB
Agent_1212bb00 (TCP-Client-ONE) CPU: 11.80% RAM: 15805.90 MB
[02:00:47] Agents: 1 | Avg CPU: 10.40% | Avg RAM: 15823.37 MB
Agent_1212bb00 (TCP-Client-ONE) CPU: 10.40% RAM: 15823.37 MB

```

FIGURE 6 – Arrêt forcé d'un client, le serveur continue sans interruption (test 4)

6.6. Arrêt du serveur

```

[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export -> stats_Apr-09-2026_1917.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 50274)
[HELLO] Agent 'Agent_bdadd5a6' (TCP-Client-ONE) registered.
[REPORT] 'Agent_bdadd5a6' -> CPU: 5.50% RAM: 15061.00 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 8.60% RAM: 15075.37 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 5.30% RAM: 15071.59 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 11.50% RAM: 15075.69 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 10.40% RAM: 15049.05 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 9.30% RAM: 15104.80 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 10.50% RAM: 14980.79 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 12.50% RAM: 14948.40 MB
[REPORT] 'Agent_bdadd5a6' -> CPU: 10.20% RAM: 15037.49 MB

[TCP SERVER] Shutting down.
C:\Users\Ferdaous\Downloads\files>

[TCP CLIENT] Starting with ID: Agent_bdadd5a6 | Hostname = TCP-Client-ONE
[TCP CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_bdadd5a6 TCP-Client-ONE'
[OK] 'REPORT Agent_bdadd5a6 1775758646 5.50000 15061.00'
[OK] 'REPORT Agent_bdadd5a6 1775758651 8.60000 15075.37'
[OK] 'REPORT Agent_bdadd5a6 1775758656 5.30000 15071.59'
[OK] 'REPORT Agent_bdadd5a6 1775758661 11.50000 15075.69'
[OK] 'REPORT Agent_bdadd5a6 1775758666 10.40000 15049.05'
[OK] 'REPORT Agent_bdadd5a6 1775758671 9.30000 15104.80'
[OK] 'REPORT Agent_bdadd5a6 1775758676 10.50000 14980.79'
[OK] 'REPORT Agent_bdadd5a6 1775758681 12.50000 14948.40'
[OK] 'REPORT Agent_bdadd5a6 1775758686 10.20000 15037.49'
[ERROR] Connection lost: [WinError 10054] An existing connection was forcibly
  closed by the remote host.
[TCP CLIENT] Server unreachable. Exiting.
[TCP CLIENT] Disconnected.
C:\Users\Ferdaous\Downloads\files>

```

FIGURE 7 – Chaque client connecté détecte la perte de connexion et s'arrête proprement

6.7. Tentative de connexion à un serveur inactif

```

[TCP CLIENT] Starting with ID: Agent_9ceee7aa | Hostname = CLIENT-NO-SERVER
[ERROR] Could not connect to 127.0.0.1:5000. Is the server running?
[TCP CLIENT] Disconnected.

C:\Users\Ferdaous\Downloads\files>

```

FIGURE 8 – Le client TCP détecte l'impossibilité d'établir la connexion et affiche un message d'erreur (test 5)

6.8. Calcul des moyennes

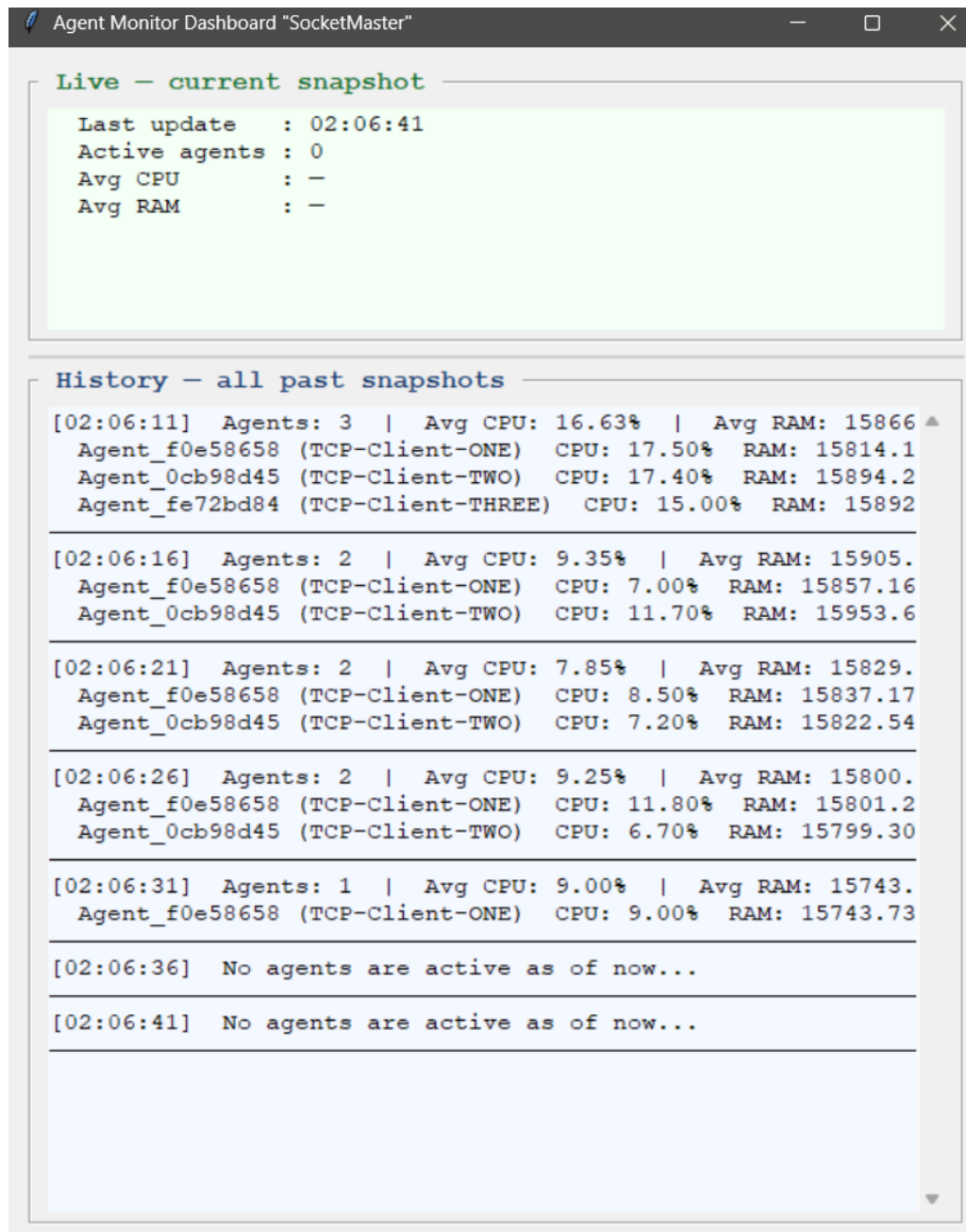


FIGURE 9 – Statistiques globales : nombre d’agents, CPU moyen, RAM moyenne

7. Difficultés Rencontrées

Délimitation des messages (\n) : Certains tests produisaient des comportements imprévisibles lorsque les messages ne se terminaient pas par un caractère de saut de ligne (\n). En effet, si deux messages arrivent presque simultanément, l’hôte ne peut pas distinguer où l’un finit et où l’autre commence. En imposant que tout message envoyé par le client ou le serveur se termine par '\n', les deux hôtes peuvent désormais recevoir des messages consécutifs de manière fiable.

Accès concurrent au dictionnaire des agents : Plusieurs threads accèdent et modifient simultanément le dictionnaire `agents`. Ce problème a été résolu en utilisant un `threading.Lock()` pour protéger toutes les opérations de lecture et d'écriture, évitant ainsi les conditions de course.

Déconnexion brutale d'un client : Lorsqu'un client se déconnecte sans envoyer de message `BYE`, le serveur lève une exception. Un bloc `try/finally` dans `handle_client` garantit que l'agent est supprimé du dictionnaire et que la connexion est fermée proprement, sans impacter les autres threads. Ce comportement diffère entre TCP et UDP : en TCP, la coupure est détectée immédiatement, tandis qu'en UDP le client continue d'émettre des paquets sans savoir que le serveur est arrêté. Ce dernier cas est géré par un mécanisme de `retry` sur le `HELLO` : jusqu'à 5 tentatives espacées de 2 secondes avant abandon.

Validation des données : Il a fallu gérer les cas où les valeurs de CPU ou RAM reçues sont hors limites ou non numériques, en retournant `ERROR InvalidData` plutôt que de laisser le programme planter.

Sécurité — vérification de propriété (ownership check) : Sans vérification, n'importe quelle connexion TCP pouvait envoyer un `REPORT` ou un `BYE` pour un agent qui ne lui appartient pas. La variable `current_agent_id` (TCP) et la table `addr_to_agent` (UDP) permettent de rejeter ces tentatives avec `ERROR Unauthorized`. Par ailleurs, un drapeau `"verified": False` est positionné à la réception du `HELLO` et ne passe à `True` qu'après le premier `REPORT` valide, ce qui empêche un attaquant d'apparaître dans le tableau de bord sans avoir prouvé son identité.

Robustesse face au flood : Sans protection, le serveur acceptait un nombre illimité d'agents et de messages `REPORT`, pouvant mener à une saturation mémoire. Deux mécanismes ont été ajoutés : une limite `MAX_AGENTS = 30` sur le nombre d'agents simultanés, et un `RATE_LIMIT_S = 2` imposant un délai minimum entre deux `REPORT` d'un même agent.

Logs illisibles lors du flood : L'affichage d'un message `[RATE LIMIT]` par requête bloquée générerait plus de 50 000 lignes de logs en quelques secondes. La solution a consisté à remplacer l'affichage immédiat par un compteur `rate_limit_count` par agent, avec un résumé affiché toutes les T secondes puis réinitialisé.

Nombres magiques : Afin de rendre le code plus lisible, moins fragile et plus facile à maintenir, toutes les constantes importantes (T la période d'envoi, l'adresse IP du serveur, `MAX_AGENTS`, etc.) sont déclarées en tête de chaque fichier Python. Cela permet de personnaliser le comportement sans avoir à modifier manuellement chaque occurrence dans le code.

Option `daemon=True` sur les threads : Sans cette option, les threads secondaires et le signal d'interruption clavier (`Ctrl+C`) entrent en conflit. `daemon=True` indique au programme que ces threads sont subordonnés au thread principal : si le processus principal est tué, les threads démons s'arrêtent automatiquement, évitant tout blocage ou boucle infinie.

Erreur `OSError: Address Already In Use` : Lors de certains tests impliquant un redémarrage du serveur, la socket précédemment utilisée n'est pas encore libérée par le système. L'option `SO_REUSEADDR` permet de réutiliser immédiatement l'adresse, supprimant cette erreur.

Interface graphique — choix de la bibliothèque : Une première approche envisageait la bibliothèque `curses` pour afficher un tableau de bord directement dans le terminal. Cette solution aurait cependant augmenté le nombre de dépendances non standard et proposé une interface moins accessible. L'utilisation de `tkinter`, inclus dans la

bibliothèque standard Python, offre une interface graphique plus lisible pour un plus large éventail d'utilisateurs.

Sécurité des threads avec Tkinter : Tkinter n'est pas thread-safe : seul le thread qui a créé la fenêtre (le thread principal) a le droit de la modifier. Toutes les mises à jour des widgets **Text** sont donc planifiées via `text_widget.after(0, callback)`, qui place le callback dans la boucle d'événements Tkinter (**mainloop**) pour qu'il soit exécuté par le thread principal au prochain cycle. Cela évite les conditions de course où plusieurs threads tenteraient d'accéder simultanément aux mêmes ressources graphiques, ce qui pourrait provoquer des problèmes de synchronisation imprévisibles.

8. Extensions Réalisées

Plusieurs extensions ont été développées pour enrichir les fonctionnalités du système.

8.1. Ajout d'un mode UDP

Le protocole UDP a été implémenté en parallèle du TCP afin de comparer les deux approches.

Critère	TCP	UDP
Connexion	Orienté connexion	Sans connexion
Fiabilité	Garantie (ACK)	Non garantie
Ordre	Garanti	Non garanti
Vitesse	Plus lent	Plus rapide
Perte de paquets	Impossible	Possible
Usage idéal	Monitoring critique	Monitoring temps réel

TABLE 3 – Comparaison TCP vs UDP pour le monitoring réseau

```

UDP-Server - python serveur x + -
[UDP SERVER] Listening on UDP port 5001 | Max agents: 30
[UDP SERVER] CSV export + statsUDP_Apr-09-2026_1939.csv
Press Ctrl+C or close the window to stop.

[HELLO] Agent 'Agent_2a909865' (UDP-Client-ONE) registered from ('127.0.0.1', 62384)
[REPORT] 'Agent_2a909865' + CPU: 4.40% RAM: 14209.76 MB
[REPORT] 'Agent_2a909865' + CPU: 17.90% RAM: 14272.52 MB
[REPORT] 'Agent_2a909865' + CPU: 8.20% RAM: 14248.28 MB
[REPORT] 'Agent_2a909865' + CPU: 4.00% RAM: 14214.29 MB

UDP-Client 1 - python client1 x + -
[UDP CLIENT] Starting with ID: Agent_2a909865 | Hostname = UDP-Client-ONE
[OK] 'HELLO Agent_2a909865 UDP-Client-ONE'
[OK] 'REPORT Agent_2a909865 1775759972 4.40000 14209.76'
[OK] 'REPORT Agent_2a909865 1775759977 17.90000 14272.52'
[OK] 'REPORT Agent_2a909865 1775759982 8.20000 14248.28'
[OK] 'REPORT Agent_2a909865 1775759987 4.00000 14214.29'
  
```

FIGURE 10 – Un seul client UDP connecté (test 7)

```

UDP-Server - python serveur
[UDP SERVER] Listening on UDP port 5801 | Max agents: 36
[UDP SERVER] CSV export -> statsUDP_Apr-09-2026_1943.csv
Press Ctrl+C or close the window to stop.

[HELLO] Agent 'Agent_06fa2886' (UDP-Client-ONE) registered from ('127.0.0.1', 56978).
[REPORT] 'Agent_06fa2886' -> CPU: 12.80% RAM: 13998.28 MB
[REPORT] 'Agent_06fa2886' -> CPU: 14.20% RAM: 14046.18 MB
[HELLO] Agent 'Agent_bb249314' (UDP-Client-THREE) registered from ('127.0.0.1', 56978).
[REPORT] 'Agent_bb249314' -> CPU: 3.80% RAM: 14040.11 MB
[REPORT] 'Agent_06fa2886' -> CPU: 19.20% RAM: 14067.12 MB
[REPORT] 'Agent_91c8d62b' -> CPU: 8.30% RAM: 14065.07 MB
[REPORT] 'Agent_bb249314' -> CPU: 5.10% RAM: 14069.68 MB
[REPORT] 'Agent_06fa2886' -> CPU: 9.10% RAM: 14044.73 MB
[REPORT] 'Agent_91c8d62b' -> CPU: 13.80% RAM: 14067.63 MB
[REPORT] 'Agent_bb249314' -> CPU: 7.90% RAM: 14045.88 MB
[REPORT] 'Agent_06fa2886' -> CPU: 5.70% RAM: 14113.21 MB
[REPORT] 'Agent_91c8d62b' -> CPU: 12.80% RAM: 14139.77 MB
[REPORT] 'Agent_bb249314' -> CPU: 5.20% RAM: 14083.11 MB
[REPORT] 'Agent_06fa2886' -> CPU: 9.60% RAM: 14073.82 MB
[REPORT] 'Agent_91c8d62b' -> CPU: 7.30% RAM: 14079.74 MB
[REPORT] 'Agent_bb249314' -> CPU: 18.40% RAM: 14068.89 MB
[REPORT] 'Agent_06fa2886' -> CPU: 7.90% RAM: 14069.72 MB
[REPORT] 'Agent_91c8d62b' -> CPU: 3.30% RAM: 14055.97 MB
[REPORT] 'Agent_bb249314' -> CPU: 3.90% RAM: 14063.97 MB

UDP-Client 1 - python client
[UDP CLIENT] Starting with ID: Agent_06fa2886 | Hostname = UDP-Client-ONE
[OK] 'HELLO Agent_06fa2886 UDP-Client-ONE'
[OK] 'REPORT Agent_06fa2886 1775760208 12.80000 13998.28'
[OK] 'REPORT Agent_06fa2886 1775760213 10.20000 14047.12'
[OK] 'REPORT Agent_06fa2886 1775760218 9.10000 14044.73'
[OK] 'REPORT Agent_06fa2886 1775760223 5.70000 14113.21'
[OK] 'REPORT Agent_06fa2886 1775760228 9.80000 14073.82'
[OK] 'REPORT Agent_06fa2886 1775760233 7.90000 14069.72'
[OK] 'REPORT Agent_06fa2886 1775760238 9.50000 14041.00'

UDP-Client 2 - python client
[UDP CLIENT] Starting with ID: Agent_91c8d62b | Hostname = UDP-Client-TWO
[OK] 'HELLO Agent_91c8d62b UDP-Client-TWO'
[OK] 'REPORT Agent_91c8d62b 1775760209 14.30000 14046.18'
[OK] 'REPORT Agent_91c8d62b 1775760214 8.30000 14065.07'
[OK] 'REPORT Agent_91c8d62b 1775760219 13.00000 14047.63'
[OK] 'REPORT Agent_91c8d62b 1775760224 12.90000 14119.77'
[OK] 'REPORT Agent_91c8d62b 1775760229 7.30000 14078.74'
[OK] 'REPORT Agent_91c8d62b 1775760234 3.30000 14058.07'
[OK] 'REPORT Agent_91c8d62b 1775760239 7.90000 14036.85'

UDP-Client 3 - python client
[UDP CLIENT] Starting with ID: Agent_bb249314 | Hostname = UDP-Client-THRE
[OK] 'HELLO Agent_bb249314 UDP-Client-THREE'
[OK] 'REPORT Agent_bb249314 1775760210 3.80000 14040.11'
[OK] 'REPORT Agent_bb249314 1775760215 5.10000 14054.68'
[OK] 'REPORT Agent_bb249314 1775760220 7.90000 14045.88'
[OK] 'REPORT Agent_bb249314 1775760225 5.20000 14083.11'
[OK] 'REPORT Agent_bb249314 1775760230 10.40000 14068.89'
[OK] 'REPORT Agent_bb249314 1775760235 3.90000 14068.97'
[OK] 'REPORT Agent_bb249314 1775760240 16.10000 14049.41'

```

FIGURE 11 – Plusieurs client UDP connectés simultanément (test 8)

```

UDP-Client - No Server
[UDP CLIENT] Starting with ID: Agent_0af5162c | Hostname = CLIENT-NO-SERVER
[WARNING] No response to HELLO, retrying (1/5)...
[WARNING] No response to HELLO, retrying (2/5)...
[WARNING] No response to HELLO, retrying (3/5)...
[WARNING] No response to HELLO, retrying (4/5)...
[WARNING] No response to HELLO, retrying (5/5)...
[ERROR] Server unreachable after several attempts. Exiting.

C:\Users\Ferdaous\Downloads\files>

```

FIGURE 12 – Tentative de connexion UDP à un serveur inactif (test 9)

```

UDP-Server
[UDP SERVER] Listening on UDP port 5801 | Max agents: 30
[UDP SERVER] CSV export -> statsUDP_Apr-09-2026_1940.csv
Press Ctrl+C or close the window to stop.

[HELLO] Agent 'Agent_02c4ab98' (UDP-Client-ONE) registered from ('127.0.0.1', 660913).
[REPORT] 'Agent_02c4ab98' -> CPU: 9.20% RAM: 14311.22 MB
[REPORT] 'Agent_02c4ab98' -> CPU: 3.70% RAM: 14284.19 MB
[REPORT] 'Agent_02c4ab98' -> CPU: 9.70% RAM: 14234.11 MB
[REPORT] 'Agent_02c4ab98' -> CPU: 7.30% RAM: 14165.40 MB

[UDP SERVER] Shutting down.

C:\Users\Ferdaous\Downloads\files>

UDP-Client 1 - python client
[UDP CLIENT] Starting with ID: Agent_02c4ab98 | Hostname = UDP-Client-ONE
[OK] 'HELLO Agent_02c4ab98 UDP-Client-ONE'
[OK] 'REPORT Agent_02c4ab98 1775760020 9.20000 14311.22'
[OK] 'REPORT Agent_02c4ab98 1775760034 3.70000 14284.19'
[OK] 'REPORT Agent_02c4ab98 1775760039 9.70000 14234.11'
[OK] 'REPORT Agent_02c4ab98 1775760044 7.30000 14165.40'
[WARNING] 'REPORT Agent_02c4ab98 1775760049 2.90000 14148.73' -> no response (packet lost?)
[WARNING] 'REPORT Agent_02c4ab98 1775760054 4.30000 14146.83' -> no response (packet lost?)
[WARNING] 'REPORT Agent_02c4ab98 1775760059 8.20000 14159.30' -> no response (packet lost?)
[WARNING] 'REPORT Agent_02c4ab98 1775760064 9.10000 14199.34' -> no response (packet lost?)
[WARNING] 'REPORT Agent_02c4ab98 1775760069 7.30000 14137.04' -> no response (packet lost?)
[WARNING] 'REPORT Agent_02c4ab98 1775760074 11.60000 14144.61' -> no response (packet lost?)

```

FIGURE 13 – Arrêt du serveur UDP en cours de session

```

TCP Server
[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-09-2026.1946.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 50340)
[HELLO] Agent 'Agent_09496fd2' (TCP-CLIENT) registered.
[REPORT] 'Agent_09496fd2' → CPU: 11.40% RAM: 14026.93 MB
[REPORT] 'Agent_09496fd2' → CPU: 6.40% RAM: 14002.81 MB
[REPORT] 'Agent_09496fd2' → CPU: 9.10% RAM: 14078.59 MB
[REPORT] 'Agent_09496fd2' → CPU: 6.00% RAM: 14096.00 MB
[REPORT] 'Agent_09496fd2' → CPU: 7.80% RAM: 14080.29 MB
[REPORT] 'Agent_09496fd2' → CPU: 6.10% RAM: 14072.20 MB

[TCP SERVER] Shutting down.
C:\Users\Ferdaous\Downloads\files>

TCP-Client
[TCP CLIENT] Starting with ID: Agent_09496fd2 | Hostname = TCP-CLIENT
[TCP CLIENT] Connected to 127.0.0.1:5000
[OK] 'HELLO Agent_09496fd2 TCP-CLIENT'
[OK] 'REPORT Agent_09496fd2 1775760375 11.40000 14026.93'
[OK] 'REPORT Agent_09496fd2 1775760380 6.40000 14002.81'
[OK] 'REPORT Agent_09496fd2 1775760385 9.10000 14078.59'
[OK] 'REPORT Agent_09496fd2 1775760390 6.00000 14096.00'
[OK] 'REPORT Agent_09496fd2 1775760395 7.80000 14080.29'
[OK] 'REPORT Agent_09496fd2 1775760400 6.10000 14072.20'
[ERROR] Connection lost: [WinError 10054] An existing connection was forcibly closed by the remote host
[TCP CLIENT] Server unreachable. Exiting.
[TCP CLIENT] Disconnected.
C:\Users\Ferdaous\Downloads\files>

UDP-Server
[UDP SERVER] Listening on UDP port 5001 | Max agents: 30
[UDP SERVER] CSV export → statsUDP_Apr-09-2026.1946.csv
Press Ctrl+C or close the window to stop.

[HELLO] Agent 'Agent_Sebddc91' (UDP-CLIENT) registered from ('127.0.0.1', 55500)
[REPORT] 'Agent_Sebddc91' → CPU: 7.10% RAM: 14003.45 MB
[REPORT] 'Agent_Sebddc91' → CPU: 6.60% RAM: 14001.69 MB
[REPORT] 'Agent_Sebddc91' → CPU: 8.30% RAM: 14080.05 MB
[REPORT] 'Agent_Sebddc91' → CPU: 8.70% RAM: 14102.36 MB
[REPORT] 'Agent_Sebddc91' → CPU: 6.30% RAM: 14082.70 MB
[REPORT] 'Agent_Sebddc91' → CPU: 4.00% RAM: 14046.25 MB
[REPORT] 'Agent_Sebddc91' → CPU: 3.30% RAM: 14009.34 MB

[UDP SERVER] Shutting down.
C:\Users\Ferdaous\Downloads\files>

UDP-Client - python clientUC
[OK] 'HELLO Agent_Sebddc91 UDP-CLIENT'
[OK] 'REPORT Agent_Sebddc91 1775760376 7.10000 14003.45'
[OK] 'REPORT Agent_Sebddc91 1775760381 6.60000 14001.69'
[OK] 'REPORT Agent_Sebddc91 1775760386 8.30000 14080.05'
[OK] 'REPORT Agent_Sebddc91 1775760391 8.70000 14102.36'
[OK] 'REPORT Agent_Sebddc91 1775760396 6.30000 14082.70'
[OK] 'REPORT Agent_Sebddc91 1775760401 4.00000 14046.25'
[OK] 'REPORT Agent_Sebddc91 1775760406 3.30000 14009.34'
[WARNING] 'REPORT Agent_Sebddc91 1775760411 5.50000 13907.06' + no response (packet lost?)
[WARNING] 'REPORT Agent_Sebddc91 1775760416 4.30000 13979.59' + no response (packet lost?)
[WARNING] 'REPORT Agent_Sebddc91 1775760421 19.20000 14065.23' + no response (packet lost?)
[WARNING] 'REPORT Agent_Sebddc91 1775760426 5.10000 14005.76' + no response (packet lost?)
[WARNING] 'REPORT Agent_Sebddc91 1775760431 5.90000 14065.16' + no response (packet lost?)
[WARNING] 'REPORT Agent_Sebddc91 1775760436 3.20000 14122.32' + no response (packet lost?)

```

FIGURE 14 – Comparaison TCP vs UDP côte à côte (test 10)

8.2. Détection d'agents inactifs

Le serveur intègre un mécanisme de détection d'inactivité. Un agent est considéré inactif lorsqu'aucun message **REPORT** n'est reçu pendant $3 \times T$ secondes. Le serveur génère alors une alerte explicite, ajoute l'agent à **inactive_alerted** pour éviter les doublons, puis le supprime du dictionnaire **agents**.

```

TCP Server - python serveurT
[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-10-2026.0208.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 58976)
[HELLO] Agent 'Agent_fdbb632d' (TCP-client-ONE) registered.
[REPORT] 'Agent_fdbb632d' → CPU: 11.20% RAM: 16160.27 MB
[REPORT] 'Agent_fdbb632d' → CPU: 13.10% RAM: 16113.29 MB
[!] Error with client ('127.0.0.1', 58976): [WinError 10054] An existing connection was forcibly closed by the remote host
[INFO] Agent 'Agent_fdbb632d' connection lost - alert in 15s if no reconnect ion.
[-] Connection closed: ('127.0.0.1', 58976)
No agents are active as of now...
No agents are active as of now...
No agents are active as of now...
[ALERT] Agent 'Agent_fdbb632d' DISCONNECTED unexpectedly (silent for 18s)
[INFO] Agent 'Agent_fdbb632d' removed from stats.
No agents are active as of now...

Agent Monitor Dashboard "SocketMaster"
Live - current snapshot
Last update : 02:09:00
Active agents : 0
Avg CPU : -
Avg RAM : -

History - all past snapshots
[02:08:35] Agents: 1 | Avg CPU: 11.20% | Avg RAM: 16160.27 MB
Agent_fdbb632d (TCP-client-ONE) CPU: 11.20% RAM: 16160.27 MB
[02:08:40] Agents: 1 | Avg CPU: 13.10% | Avg RAM: 16113.29 MB
Agent_fdbb632d (TCP-client-ONE) CPU: 13.10% RAM: 16113.29 MB
[02:08:45] No agents are active as of now...
[02:08:50] No agents are active as of now...
[02:08:55] No agents are active as of now...
[02:09:00] No agents are active as of now...

```

FIGURE 15 – Le serveur détecte et affiche une alerte pour un agent inactif (test 4)

8.3. Export des statistiques en CSV

À chaque cycle de statistiques, un snapshot est automatiquement ajouté au fichier **stats_<horodatage>.csv** (TCP) ou **statsUDP_<horodatage>.csv** (UDP), avec les colonnes : **timestamp**, **agent_id**, **hostname**, **cpu_pct**, **ram_mb**. Seuls les agents **verified = True** sont exportés.

	stats.csv > data
1	2026-03-26 00:21:58.731733,agent_d9818656,Ferouzette,0.0,611.91
2	2026-03-26 00:22:03.741388,agent_d9818656,Ferouzette,0.0,612.07
3	2026-03-26 00:22:08.767511,agent_d9818656,Ferouzette,0.0,612.07
4	2026-03-26 00:22:13.300397,agent_d9818656,Ferouzette,0.5,634.19
5	2026-03-26 00:22:18.328616,agent_d9818656,Ferouzette,0.0,637.73
6	2026-03-26 00:22:23.381262,agent_d9818656,Ferouzette,0.1,637.73
7	2026-03-26 00:22:28.415567,agent_d9818656,Ferouzette,0.0,637.73
8	2026-03-26 00:22:33.431097,agent_d9818656,Ferouzette,0.0,643.84
9	2026-03-26 00:22:33.431132,agent_6544d20f,Ferouzette,0.0,643.84
10	2026-03-26 00:22:38.444024,agent_d9818656,Ferouzette,0.0,644.11
11	2026-03-26 00:22:38.444175,agent_6544d20f,Ferouzette,0.0,644.11
12	2026-03-26 00:22:38.444187,agent_614007c5,Ferouzette,0,0
13	2026-03-26 00:22:42.964453,agent_d9818656,Ferouzette,0.0,651.16
14	2026-03-26 00:22:42.964490,agent_6544d20f,Ferouzette,0.2,651.16
15	2026-03-26 00:22:42.964499,agent_614007c5,Ferouzette,0.1,650.44

FIGURE 16 – Contenu du fichier stats.csv généré automatiquement

8.4. Ajout d'un identifiant unique UUID

Chaque agent est identifié par un UUID généré via le module uuid de Python. Deux options sont disponibles au lancement :

- **Option 1** : passer un nom fixe en argument :

```
python clientTCP.py MonHostname
python clientUDP.py MonHostname
```

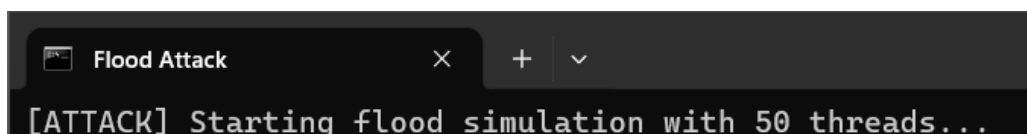
- **Option 2** : un UUID aléatoire est généré automatiquement : Agent_ suivi de 8 caractères (ex. Agent_fb0e3f14) :

```
python clientTCP.py
python clientUDP.py
```

Ce choix est **intentionnellement non persistant** : un nouvel identifiant est créé à chaque démarrage, ce qui apporte un avantage de sécurité — même si un attaquant intercepte l'ID d'une session, cet ID sera invalide à la prochaine connexion. Plusieurs agents peuvent tourner simultanément sur la même machine (utile en environnement WSL).

8.5. Simulation d'attaque — Envoi massif de REPORT

Un script **flood.py** a été créé pour simuler une attaque par déni de service en lançant **50 threads simultanés**, chacun envoyant **1 000 messages REPORT en rafale**. Cela permet de tester la robustesse du serveur sous une charge élevée.

FIGURE 17 – Lancement de l'attaque flood (**test 6**)


```

TCP-Server
[-] Connection closed: ('127.0.0.1', 50993)
[BLOCK] Agent limit (30) reached. 'agent_867c3615' refused.
[-] Connection closed: ('127.0.0.1', 50993)
[BLOCK] Agent limit (30) reached. 'agent_955da926' refused.
[-] Connection closed: ('127.0.0.1', 50999)
[BLOCK] Agent limit (30) reached. 'agent_1e9256ea' refused.
[-] Connection closed: ('127.0.0.1', 50995)
[REPORT] 'agent_0b5a1d8e' + CPU: 0.00% RAM: 10736.71 MB
[REPORT] 'agent_2fb3aeed' + CPU: 0.00% RAM: 15676.82 MB
[BLOCK] Agent limit (30) reached. 'agent_ba562b33' refused.
[BLOCK] Agent limit (30) reached. 'agent_7a278882' refused.
[-] Connection closed: ('127.0.0.1', 50996)
[-] Connection closed: ('127.0.0.1', 50998)
[BLOCK] Agent limit (30) reached. 'agent_aaebd874' refused.
[BLOCK] Agent limit (30) reached. 'agent_6756856c' refused.
[-] Connection closed: ('127.0.0.1', 50999)
[BLOCK] Agent limit (30) reached. 'agent_f6e6cbab' refused.
[-] Connection closed: ('127.0.0.1', 51000)
[REPORT] 'agent_4487543c' + CPU: 3.11% RAM: 14718.84 MB
[-] Connection closed: ('127.0.0.1', 51000)
[REPORT] 'agent_c541db24' + CPU: 3.29% RAM: 14688.24 MB
[BLOCK] Agent limit (30) reached. 'agent_17888a97' refused.
[BLOCK] Agent limit (30) reached. 'agent_268b5256' refused.
[-] Connection closed: ('127.0.0.1', 51001)
[REPORT] 'agent_16a566ac' + CPU: 0.26% RAM: 10664.03 MB
[-] Connection closed: ('127.0.0.1', 51002)
[BLOCK] Agent limit (30) reached. 'agent_321ed669' refused.
[BLOCK] Agent limit (30) reached. 'agent_68876bb9' refused.
[-] Connection closed: ('127.0.0.1', 51004)
[BLOCK] Agent limit (30) reached. 'agent_ec7234d5' refused.
[-] Connection closed: ('127.0.0.1', 51007)
[REPORT] 'agent_328c71c2' + CPU: 0.00% RAM: 10734.25 MB
[-] Connection closed: ('127.0.0.1', 51005)
[BLOCK] Agent limit (30) reached. 'agent_70181d49' refused.
[REPORT] 'agent_787c80fa' + CPU: 0.00% RAM: 10727.76 MB
[-] Connection closed: ('127.0.0.1', 51003)
[BLOCK] Agent limit (30) reached. 'agent_3f12158a' refused.
[BLOCK] Agent limit (30) reached. 'agent_0848e556' refused.
[-] Connection closed: ('127.0.0.1', 50997)
[BLOCK] Agent limit (30) reached. 'agent_07a1dd75' refused.
[-] Connection closed: ('127.0.0.1', 51000)

Flood Attack
[ATTACK] Starting Flood simulation with 50 threads...
[agent_2656da77] Rejected by server: ERROR Max agents reached
[agent_071c7152] Rejected by server: ERROR Max agents reached
[agent_867c3615] Rejected by server: ERROR Max agents reached
[agent_99daa426] Rejected by server: ERROR Max agents reached
[agent_1e9256ea] Rejected by server: ERROR Max agents reached
[agent_ba622631] Rejected by server: ERROR Max agents reached
[agent_7a278882] Rejected by server: ERROR Max agents reached
[agent_aaebd874] Rejected by server: ERROR Max agents reached
[agent_6756856c] Rejected by server: ERROR Max agents reached
[agent_f6e6cbab] Rejected by server: ERROR Max agents reached
[agent_17888a97] Rejected by server: ERROR Max agents reached
[agent_268b5256] Rejected by server: ERROR Max agents reached
[agent_121ed468] Rejected by server: ERROR Max agents reached
[agent_68876bb9] Rejected by server: ERROR Max agents reached
[agent_3f12158a] Rejected by server: ERROR Max agents reached
[agent_0848e556] Rejected by server: ERROR Max agents reached
[agent_07a1dd75] Rejected by server: ERROR Max agents reached
[agent_199ec2f1] Rejected by server: ERROR Max agents reached
[ATTACK] Flood simulation complete.
C:\Users\Verdous\Downloads\Files>

```

FIGURE 18 – Enregistrement des 30 premiers agents seulement (MAX_AGENTS = 30)

```

[RATE LIMIT] Blocked request summary:
- agent_886a7559: 998 blocked request(s)
- agent_ae9370f2: 998 blocked request(s)
- agent_a9e243d8: 998 blocked request(s)
- agent_c894ea5a: 998 blocked request(s)
- agent_a8fe0efb: 998 blocked request(s)
- agent_342831e3: 998 blocked request(s)
- agent_fa72e762: 998 blocked request(s)
- agent_7b32da5f: 998 blocked request(s)
- agent_0da38b46: 998 blocked request(s)
- agent_9b492559: 998 blocked request(s)
- agent_71d3e886: 998 blocked request(s)
- agent_b0915612: 998 blocked request(s)
- agent_0c102332: 998 blocked request(s)
- agent_789cb414: 998 blocked request(s)
- agent_56d202d9: 998 blocked request(s)
- agent_9886326f: 998 blocked request(s)
- agent_986d5c47: 998 blocked request(s)
- agent_b81b4958: 998 blocked request(s)
- agent_c014c2ae: 998 blocked request(s)
- agent_d4b6d6bc: 998 blocked request(s)
- agent_580b0a3f: 998 blocked request(s)
- agent_c62f85d4: 998 blocked request(s)
- agent_124bea09: 998 blocked request(s)
- agent_d05a1d0e: 998 blocked request(s)
- agent_2fb3aeed: 998 blocked request(s)
- agent_4487543c: 998 blocked request(s)
- agent_c541db24: 998 blocked request(s)
- agent_16a566ac: 998 blocked request(s)
- agent_328c71c2: 998 blocked request(s)
- agent_787c80fa: 998 blocked request(s)

```

FIGURE 19 – Le serveur bloque les requêtes envoyées trop rapidement et affiche un résumé du nombre de REPORT rejetés par agent


```

TCP-Server
[INFO] Agent 'agent_886a7559' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50961)
[INFO] Agent 'agent_a9e243d8' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50962)
[INFO] Agent 'agent_ae9370f2' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50960)
[INFO] Agent 'agent_c894ea5a' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50964)
[INFO] Agent 'agent_342831e3' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50965)
[INFO] Agent 'agent_a8fe6e1b' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50963)
[INFO] Agent 'agent_7b32da5f' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50968)
[INFO] Agent 'agent_9b492559' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50967)
[INFO] Agent 'agent_fa72e762' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50966)
[INFO] Agent 'agent_0da38b46' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50969)
[INFO] Agent 'agent_0c102332' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50972)
[INFO] Agent 'agent_9886326f' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50970)
[INFO] Agent 'agent_71d3e886' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50978)
[INFO] Agent 'agent_b0915612' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50973)
[INFO] Agent 'agent_124bea09' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50976)
[INFO] Agent 'agent_789cb414' cleaned up on disconnect.
[INFO] Agent 'agent_c014c2ae' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50980)
[INFO] Agent 'agent_d05a1d0e' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50979)
[INFO] Agent 'agent_c62f85d4' cleaned up on disconnect.
[INFO] Agent 'agent_c511ab21' cleaned up on disconnect.
[INFO] Agent 'agent_56d282d9' cleaned up on disconnect.
[INFO] Agent 'agent_d4b6d6bc' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 50984)
[INFO] Agent 'agent_580b8a3f' cleaned up on disconnect.

Flood Attack
[ATTACK] Starting flood simulation with 50 threads...
[agent_2e560a47] Rejected by server: ERROR Max agents reached
[agent_e971c715] Rejected by server: ERROR Max agents reached
[agent_867c3615] Rejected by server: ERROR Max agents reached
[agent_995da426] Rejected by server: ERROR Max agents reached
[agent_1e9256a1] Rejected by server: ERROR Max agents reached
[agent_ba662263] Rejected by server: ERROR Max agents reached
[agent_7a278082] Rejected by server: ERROR Max agents reached
[agent_aaebd074] Rejected by server: ERROR Max agents reached
[agent_6f75850c] Rejected by server: ERROR Max agents reached
[agent_f6a6cb36] Rejected by server: ERROR Max agents reached
[agent_17588e79] Rejected by server: ERROR Max agents reached
[agent_206b5256] Rejected by server: ERROR Max agents reached
[agent_121ed460] Rejected by server: ERROR Max agents reached
[agent_689f6089] Rejected by server: ERROR Max agents reached
[agent_ec723d45] Rejected by server: ERROR Max agents reached
[agent_75381d0e] Rejected by server: ERROR Max agents reached
[agent_3f12150a] Rejected by server: ERROR Max agents reached
[agent_0848e556] Rejected by server: ERROR Max agents reached
[agent_4711dd76] Rejected by server: ERROR Max agents reached
[agent_199ec2f1] Rejected by server: ERROR Max agents reached
[ATTACK] Flood simulation complete.
C:\Users\Ferdaous\Downloads\files>

```

FIGURE 20 – Fin du flood

8.5.1. Analyse du comportement du serveur face au flood

Le déroulement du flood se décompose en trois phases :

1. 50 threads lancés simultanément, chacun envoie un **HELLO** ; le serveur enregistre les premiers 30 agents seulement grâce à **MAX_AGENTS = 30**, les 20 suivants reçoivent **ERROR Max agents reached**.
2. Chaque agent envoie 1000 messages **REPORT** en rafale → la majorité est bloquée par le rate-limiting ; seul 1 **REPORT** est accepté toutes les **RATE_LIMIT_S = 2** secondes par agent.
3. Le script **flood.py** termine ses 1000 itérations, ferme le socket via **sock.close()**. Le serveur détecte la fermeture TCP et affiche **[-] Connection closed**

Message	Signification
[HELLO]	Agent enregistré normalement
[BLOCK] Max agents reached	Plafond MAX_AGENTS atteint, agent refusé
[RATE LIMIT]	Requête bloquée car envoyée trop rapidement
[INFO] ... cleaned up	Agent supprimé proprement du dictionnaire
[-] Connection closed	flood.py a terminé et fermé le socket proprement

TABLE 4 – Messages du serveur lors de la simulation de flood

Le serveur a correctement résisté au flood grâce aux mécanismes de rate-limiting et de limitation du nombre d'agents, sans crash ni perte de contrôle.

8.6. Sécurité — Ownership check

Un mécanisme d'ownership check a été implémenté pour garantir qu'une connexion ne peut envoyer des commandes **REPORT** ou **BYE** que pour l'agent qu'elle a elle-même

enregistré. En TCP, cela repose sur la variable `current_agent_id` associée à chaque thread client ; en UDP, la table `addr_to_agent` assure la même protection en liant chaque adresse source à un unique agent.

```

[TCP SERVER] Listening on port 5000 | Max agents: 30
[TCP SERVER] CSV export → stats_Apr-09-2026_1925.csv
Press Ctrl+C (or close the window) to stop.

[+] Connection from ('127.0.0.1', 55793)
[HELLO] Agent 'Agent_6a3316bf' (TCP-Client-ONE) registered.
[REPORT] 'Agent_6a3316bf' → CPU: 16.20% RAM: 15000.15 MB
[REPORT] 'Agent_6a3316bf' → CPU: 9.50% RAM: 14993.24 MB
[REPORT] 'Agent_6a3316bf' → CPU: 7.00% RAM: 14981.46 MB
[REPORT] 'Agent_6a3316bf' → CPU: 5.38% RAM: 15009.20 MB
[REPORT] 'Agent_6a3316bf' → CPU: 20.00% RAM: 15030.45 MB
[REPORT] 'Agent_6a3316bf' → CPU: 7.60% RAM: 14950.33 MB
[REPORT] 'Agent_6a3316bf' → CPU: 5.10% RAM: 14898.92 MB
[REPORT] 'Agent_6a3316bf' → CPU: 2.90% RAM: 14980.77 MB
[REPORT] 'Agent_6a3316bf' → CPU: 3.60% RAM: 14899.41 MB
[REPORT] 'Agent_6a3316bf' → CPU: 6.10% RAM: 14911.03 MB
[+] Connection from ('127.0.0.1', 52277)
[HELLO] Agent 'Agent_9eacebca' (IMPOSTER) registered.
[SECURITY] ('127.0.0.1', 52277) tried to REPORT for 'Agent_6a3316bf' but o
wns 'Agent_9eacebca'. Rejected.
[REPORT] 'Agent_6a3316bf' → CPU: 4.00% RAM: 14906.91 MB
[SECURITY] ('127.0.0.1', 52277) tried to REPORT for 'Agent_6a3316bf' but o
wns 'Agent_9eacebca'. Rejected.
[REPORT] 'Agent_6a3316bf' → CPU: 4.10% RAM: 14903.47 MB
[SECURITY] ('127.0.0.1', 52277) tried to REPORT for 'Agent_6a3316bf' but o
wns 'Agent_9eacebca'. Rejected.
[REPORT] 'Agent_6a3316bf' → CPU: 4.20% RAM: 14898.68 MB
[SECURITY] ('127.0.0.1', 52277) tried to REPORT for 'Agent_6a3316bf' but o
wns 'Agent_9eacebca'. Rejected.
[REPORT] 'Agent_9eacebca' → CPU: 6.00% RAM: 14914.04 MB
[SECURITY] ('127.0.0.1', 52277) tried to REPORT for 'Agent_6a3316bf' but o
wns 'Agent_9eacebca'. Rejected.
[INFO] Agent 'Agent_9eacebca' cleaned up on disconnect.
[-] Connection closed: ('127.0.0.1', 52277)
[REPORT] 'Agent_6a3316bf' → CPU: 14.00% RAM: 14988.71 MB
[REPORT] 'Agent_6a3316bf' → CPU: 6.60% RAM: 14950.19 MB

```

FIGURE 21 – Résultat du test d'ownership

9. Conclusion

Ce mini-projet a permis de mettre en pratique les concepts fondamentaux des systèmes distribués : communication par sockets TCP et UDP, protocole applicatif personnalisé, gestion des threads, robustesse face aux erreurs et aux attaques. Le système implémenté est fonctionnel et répond à toutes les exigences du cahier des charges.

Les extensions réalisées — mode UDP complet, export CSV horodaté, UUID dynamique, détection d'agents inactifs, mécanismes de sécurité (ownership check, rate-limiting, plafond d'agents), tableau de bord Tkinter et simulation d'attaque flood — démontrent la capacité du système à évoluer vers un environnement de production réel. La suite de tests automatisée `run_tests.bat` offre une démonstration complète des 10 scénarios sans aucune configuration préalable.